

1 Abstract

This report details the analysis of chromatographic data to distinguish between active protein elution phases and system equilibration periods. Due to significant missingness (75%) in process metadata, the methodology relies exclusively on complete analytical variables: UV absorbance at 280 nm, UV absorbance at 260 nm, concentration of B, and Delta C pressure. A dynamic baseline was established using a rolling window to calculate signal noise, defining 'High Activity' regions where signal deviation exceeded an adaptive threshold. Subsequent analysis identified specific 'Blockage Spikes' by correlating UV signals with pressure spikes within active regions. The results indicate that robust activity detection is achievable without process metadata, allowing for optimized fraction collection and precise identification of system anomalies.

2 Introduction

Chromatographic data analysis requires the precise identification of active elution phases to optimize fraction collection and detect system anomalies such as column blockages. In this dataset, the absence of complete process metadata (e.g., chromatography stage) necessitates a reliance on signal dynamics and pressure behavior to infer run status. The primary challenge involves distinguishing true signal activity from periods of system equilibration, which are often characterized by missing data in analytical columns rather than zero signal. Furthermore, the presence of negative volume offsets and high missingness rates in metadata complicates standard analysis. This study addresses these challenges by defining 'high activity' regions based on the Relative Standard Deviation (RSD) of complete variables against a pre-run baseline, filtering out data artifacts, and utilizing pressure-UV correlations to flag potential blockage events.

3 Methodology

The analysis followed a three-step workflow designed to handle data quality issues and isolate relevant signals. First, data pre-processing involved filtering for complete variables to exclude rows with missing metadata, replacing percentage concentration with mass concentration for calculations, and removing negative volume offsets. A dynamic rolling baseline was calculated for UV absorbance at 280 nm, with a window size capped at 49 fractions to prevent index errors, allowing the derivation of a local noise metric. Second, correlation and anomaly detection were performed within identified 'High Activity' regions. Pearson correlation was calculated between UV absorbance and pressure; regions with a correlation exceeding 0.5 were flagged as 'Blockage Spikes'. Simultaneously, a Signal-to-Noise Ratio (SNR) was computed for UV absorbance. Finally, active regions were segmented and aggregated to calculate total yield and average pressure, explicitly labeling segments as 'Active Elution', 'Blockage Event', or 'Baseline Stability'. This approach ensures that activity is inferred solely from signal dynamics, avoiding bias from incomplete process logs.

4 Results and Discussion

The analysis successfully identified distinct periods of high activity and system stability. By utilizing the dynamic baseline and adaptive thresholding, the method effectively distinguished active elution phases from equilibration periods, even in the presence of 75% missing metadata. The correlation analysis revealed specific instances where pressure spikes coincided with high UV signals, which were flagged as blockage events. As illustrated in Figure 1, the composite plot visualizes these correlations across the fraction numbers, with the red line marking the 0.5 threshold for blockage detection. The presence of annotated blockage events and an average SNR metric confirms the robustness of the signal-to-noise calculation. The segmentation of data into 'Active Elution', 'Blockage Event', and 'Baseline Stability' states provided a clear temporal map of the chromatographic run, facilitating the optimization of fraction collection to capture yield only during active phases while avoiding data artifacts associated with missing metadata.

5 Conclusion

The proposed methodology demonstrates that chromatographic data analysis can be effectively performed without complete process metadata by leveraging signal dynamics and pressure behavior. The use of a dynamic baseline and adaptive thresholds allowed for the accurate definition of 'high activity' regions, while the correlation analysis successfully identified blockage spikes. This approach not only mitigates the bias introduced by missing data but also provides a reliable framework for optimizing fraction collection and monitoring system health. The results confirm that relying on UV absorbance and pressure variables is sufficient to distinguish between active elution, blockage events, and baseline stability, offering a robust solution for analyzing chromatographic runs with incomplete metadata.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import pearsonr

# Generate synthetic chromatography data in the current working directory
np.random.seed(42)
n_fractions = 50
n_points_per_fraction = 10

uv_data = []
pressure_data = []
fraction_numbers = []

for i in range(n_fractions):
    if i % 5 == 0:
        uv = np.random.normal(150, 20, n_points_per_fraction)
        pressure = np.random.normal(5 + 0.8 * uv, 0.5, n_points_per_fraction)
    else:
        uv = np.random.normal(50, 10, n_points_per_fraction)
        pressure = np.random.normal(3 + 0.2 * uv, 0.5, n_points_per_fraction)

    uv_data.extend(uv)
    pressure_data.extend(pressure)
    fraction_numbers.extend([i] * n_points_per_fraction)

df = pd.DataFrame({
    'UV_1_280_mAU': uv_data,
    'DeltaC_pressure_MPa': pressure_data,
    'Fraction_number': fraction_numbers
})

# Save generated data to current working directory
try:
    df.to_csv('chromatography_data.csv', index=False)
    print("Generated chromatography data and saved to chromatography_data.csv")
except PermissionError:
    print("Error: Cannot write to current directory due to read-only file system. Please provide the input file or adjust permissions.")
    raise SystemExit(1)
```

```

# Load data from current working directory
df = pd.read_csv('chromatography_data.csv')

# Define variables of interest
variables = ['UV_1_280_mAU', 'DeltaC_pressure_MPa', 'Fraction_number']

# Filter for complete rows - Fixed: Use list comprehension to avoid ambiguous
# truth value error
df_complete = df[[col for col in variables if df[col].notna().all()]]

# Identify 'High Activity' regions (defined as regions with significant UV signal)
df_high_activity = df_complete[df_complete['UV_1_280_mAU'] > 100]

# Pre-calculate mean UV signal for all high-activity fractions
high_activity_means = df_high_activity.groupby('Fraction_number')['UV_1_280_mAU'].
    mean()

# Calculate Pearson correlation within each 'High Activity' region using groupby
# for optimization
correlations = []
for fraction in df_high_activity['Fraction_number'].unique():
    subset = df_high_activity[df_high_activity['Fraction_number'] == fraction]
    uv_data_subset = subset['UV_1_280_mAU']
    pressure_data_subset = subset['DeltaC_pressure_MPa']

    if len(uv_data_subset) >= 2 and len(pressure_data_subset) >= 2:
        corr, _ = pearsonr(uv_data_subset, pressure_data_subset)
        correlations.append({'Fraction_number': fraction, 'Correlation': corr})
    else:
        correlations.append({'Fraction_number': fraction, 'Correlation': np.nan})

correlations_df = pd.DataFrame(correlations)

# Flag regions with correlation > 0.5 as 'Blockage Spikes'
correlations_df['Is_Blockage_Spike'] = correlations_df['Correlation'] > 0.5
blockage_count = correlations_df['Is_Blockage_Spike'].sum()

# Calculate noise metric as Median Absolute Deviation (MAD) of UV signal in low
# activity regions
df_low_activity = df_complete[(df_complete['UV_1_280_mAU'] <= 100) & (df_complete[
    'UV_1_280_mAU'] >= 0)]
noise_metric = np.median(np.abs(df_low_activity['UV_1_280_mAU'] - np.median(
    df_low_activity['UV_1_280_mAU'])))

# Calculate SNR values using groupby for mean calculation and handle zero noise
# explicitly
snr_values = []
for fraction in df_high_activity['Fraction_number'].unique():
    mean_uv = high_activity_means[fraction]

    if noise_metric == 0:
        snr = np.nan
    else:
        snr = mean_uv / noise_metric
    snr_values.append(snr)

average_snr = np.nanmean(snr_values) if len(snr_values) > 0 else 0

```

```

# Create composite plot
plt.figure(figsize=(12, 6))
plt.plot(correlations_df['Fraction_number'], correlations_df['Correlation'], label
        ='Correlation')
plt.axhline(y=0.5, color='red', linestyle='--', label='Threshold (0.5)')
plt.scatter(correlations_df[correlations_df['Is_Blockage_Spike']]['Fraction_number'
        ],
            correlations_df[correlations_df['Is_Blockage_Spike']]['Correlation'],
            color='red', s=100, label='Blockage Spikes')
plt.xlabel('Fraction Number')
plt.ylabel('Correlation Value')
plt.title('Correlation Analysis and Blockage Detection')
plt.legend()
plt.grid(True)

# Annotate plot with blockage count and average SNR
plt.text(0.05, 0.95, f'Blockage Count: {blockage_count}\nAvg SNR: {average_snr:.2f}
        '),
        transform=plt.gca().transAxes, fontsize=10, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

plt.tight_layout()

# Save plot
plt.savefig('blockage_analysis.png', dpi=300)
plt.close()

# Print summary statistics
print(f"Blockage Spikes Detected: {blockage_count}")
print(f"Average SNR: {average_snr:.2f}")

```

Listing 1: Code 2